SOFTWARE REQUIREMENTS SPECIFICATION

for

SwiftStock - AI-Based Predictive Inventory and Supply Chain Management System

Prepared by	L. Nithish Kumar Goud (160124733327) A. Veeren Reddy (160124733312)
Institution	CBIT — Department of Computer Science
Instructor	D. Meenakshi
Course	Software Engineering — Lab Batch-2

Table of Contents

1. Introduction **3**

1.1 Document Purpose 3

1.2 Product Scope 3

1.3 Definitions, Acronyms, and Abbreviations 4

1.4 References 4

1.5 Overview 4

2. General Description **4**

2.1 Product Perspective 4

2.2 Product Functions 5

2.3 User Characteristics 5

2.4 General Constraints 5

2.5 Assumptions and Dependencies 6

3. Specific Requirements **7**

3.1 Functional Requirements 7

3.2 External Interface Requirements 8

3.3 Performance Requirements 9

3.4 Design Constraints 10

3.6 Other Requirements 10

4. Appendices **11**

Appendix A — Database Entity Schema (Prisma) 11

Appendix B — Project Directory Structure 11

Appendix C — System Use Case Workflows 11

5. Index **12**

1. Introduction

1.1 Document Purpose

This Software Requirements Specification (SRS) provides a complete, unambiguous, and testable description of SwiftStock, an AI-augmented SaaS inventory and supply-chain management platform. It defines the functional and non-functional behaviour of the system, its architectural constraints, external interfaces, and the integration of Large Language Model (LLM) intelligence into demand forecasting and procurement, so that developers, testers, and evaluators share a single source of truth for what is being built.

This document is intended to be read by the following audiences:

- **Software Developers & AI Coding Agents —** to implement the Next.js frontend, Server Action/API backend, Prisma schema, and AI integration exactly as specified.
- **QA Engineers —** to derive test cases directly from the functional requirements (FR-x.x) and acceptance criteria.
- **Academic Evaluators —** to assess the depth of architectural planning and adherence to software engineering practice.
- **Project Stakeholders —** to confirm the delivered product matches the agreed scope and business rules.

1.2 Product Scope

SwiftStock is a dashboard-first, multi-tenant SaaS platform that replaces spreadsheet- and legacy-ERP-based inventory tracking with a near-zero-latency web application, augmented by an AI prediction layer that continuously reasons about stock velocity and supplier replenishment.

In Scope:

Secure authentication and organization management (Clerk); strict multi-tenant data isolation; full product-catalog CRUD; real-time stock-transaction logging; configurable low-stock alerting with email and webhook dispatch; manager-approval workflows for non-admin stock changes; barcode management (EAN/UPC/QR); CSV import/export; category management; a full audit trail; charted analytics dashboards; Freemium/PRO subscription billing via Razorpay; authenticated file uploads via UploadThing; developer-facing webhook integrations; and an AI subsystem that performs LLM-based demand forecasting, dynamic restock-threshold calculation, automated supplier email drafting, and autonomous purchase-order generation.

Out of Scope:

Multi-warehouse geographic tracking, real-time WebSocket collaboration, third-party sales-channel integrations (Shopify, Amazon), and direct shipping-carrier APIs (FedEx/UPS) are explicitly excluded from the current release.

The guiding objective is a Server-First architecture (Next.js 16 App Router + React Server Components) over a PostgreSQL/Prisma data layer, in which an isolated AI module continuously augments — but never blocks — core inventory operations.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SaaS	Software as a Service — centrally hosted application accessed by subscription.
SRS	Software Requirements Specification, this document.
RBAC	Role-Based Access Control — Admin vs. Member permissions within an organization.
RSC	React Server Component — renders exclusively on the server for smaller client bundles.
ORM	Object-Relational Mapping; Prisma is the ORM used in this project.

Term	Definition
SKU	Stock Keeping Unit — unique product identifier, enforced per-organization.
CUID	Collision-resistant Unique Identifier — default primary-key format.
LLM	Large Language Model (OpenAI GPT-4o / Groq Llama) used for forecasting and drafting.
PO	Purchase Order — a procurement record sent to a supplier.
HMAC	Hash-based Message Authentication Code (SHA-256) used for webhook verification.
CRUD	Create, Read, Update, Delete.
CSV	Comma-Separated Values — format used for bulk import/export.
Multi-Tenancy	Architecture in which one deployment serves many isolated organizations.
Dynamic Threshold	AI-calculated reorder point that replaces a static low-stock threshold.
Clerk	Third-party authentication and organization-management platform.
Prisma	Type-safe Node.js/TypeScript ORM for PostgreSQL.
Razorpay	Payment gateway used for PRO-tier subscription billing.
Resend	Transactional email API used for alerts and PO emails.
Zustand / Zod	Client-state library / schema-validation library used respectively for UI state and input validation.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Next.js v16 Documentation — App Router, React Server Components, Server Actions.
- React v19 Documentation — Hooks architecture and Strict Mode.
- Prisma v7 Documentation — PostgreSQL ORM, migrations, and the @prisma/adapter-pg driver.
- Clerk Documentation — Authentication, Organizations, Webhooks, RBAC middleware.
- Razorpay Subscription API Documentation — recurring billing and webhook lifecycle.
- Resend API Documentation — transactional email dispatch.
- OpenAI API Documentation — chat completions and strict JSON-mode responses.
- Groq API Documentation — high-speed LLM inference, OpenAI-SDK compatible.

1.5 Overview

Section 2 gives a high-level description of the product, its users, and its constraints. Section 3 specifies exhaustive functional and non-functional requirements. Section 4 supplies supporting appendices — the database schema, directory layout, and use-case workflows. Section 5 is an alphabetical index of key terms and where they are discussed.

2. General Description

2.1 Product Perspective

SwiftStock is an independent, self-contained, multi-tenant SaaS application built on a Server-First architecture. Its AI Logic Tier integrates LLMs into the inventory pipeline for predictive analytics and autonomous procurement. The system is organized into seven cooperating tiers:

Tier	Responsibility
Presentation (Client)	React 19 + Next.js 16 App Router; hybrid RSC/Client Components; Zustand for UI state; TanStack Query v5 for server-state caching; Tailwind CSS v4 styling; Geist font.

Tier	Responsibility
Logic & AI (Server)	Server Actions ("use server") for mutations; API Routes for external integrations; isolated AI module (src/lib/ai/) wrapping the OpenAI/Groq SDK; Zod validation on every input; Upstash-Redis sliding-window rate limiting.
Data (Database)	PostgreSQL accessed through Prisma v7 with the @prisma/adapter-pg driver; 15 models under strict multi-tenant isolation with indexed foreign keys and composite unique constraints.
Auth & Identity	Clerk handles sign-up/in, sessions, organizations, and RBAC; Svix-verified webhooks synchronize identity data into PostgreSQL in real time.
Email Service	Resend dispatches low-stock alerts, AI-drafted purchase-order emails, and a daily digest.
Payment	Razorpay processes FREE→PRO upgrades with HMAC-verified webhooks for subscription lifecycle events.
File Upload	UploadThing manages authenticated image uploads (4 MB limit) for product photos and organization logos.

2.2 Product Functions

The system automates and secures the following core workflows:

Function	Summary
Identity & Access Management	Clerk-hosted registration/login; webhook-synced user, organization, and membership records.
Multi-Tenant Isolation	Every query is scoped to a server-derived organization ID; client-supplied tenant IDs are never trusted.
Product Catalog Management	Full CRUD with organization-scoped SKUs, categories, and UploadThing-hosted images.
Stock Transaction Management	Immutable stock-in/out ledger; atomic transactions prevent negative stock.
Manager Approval Workflow	Non-admin adjustments are queued as PENDING and require admin approval or rejection.
Low-Stock Alert System	Configurable per-product thresholds trigger email and webhook notifications; alerts auto-resolve on restock.
AI Demand Forecasting	A daily cron feeds 30-day sales velocity to an LLM, which returns a new dynamic threshold with reasoning and a confidence score.
Autonomous Procurement	When stock falls under the AI threshold, a draft Purchase Order and supplier email are generated automatically.
Supplier Management	CRUD for supplier contact profiles linked to products for automated replenishment.
Subscription Billing	Freemium model (FREE: 3 products / 15 tx/day; PRO: unlimited) via Razorpay checkout and webhooks.
Analytics Dashboard	KPI widgets, a 30-day stock-movement chart, and a recent-activity feed.
CSV Import/Export	Bulk inventory and transaction import with SKU-based upsert; PDF/CSV export.
Audit Trail	Every product mutation is logged with actor, action, entity, and a JSON diff.
Developer Webhooks	Organizations register endpoints that receive HMAC-signed event payloads (e.g. stock.low).
Organization Settings	Profile, logo, currency, and member-roster management synced from Clerk.

2.3 User Characteristics

Access is governed by a Clerk-managed RBAC model enforced at the organization level.

Role	Characteristics & Permissions
Administrator (org:admin)	High technical literacy; full product CRUD; direct stock adjustments; approves/rejects member requests; manages categories, webhooks, barcodes, billing, and suppliers; approves AI-drafted purchase orders; can trigger manual AI forecasts.
Member (default)	General contributor; read access to dashboard, inventory, transactions, and audit logs; stock changes are routed to the approval queue; read-only visibility into suppliers and purchase orders.

Role	Characteristics & Permissions
Personal Workspace User	Individual operating without a team; admin-equivalent access inside an auto-provisioned personal workspace; adjustments apply immediately; subject to FREE-tier limits unless upgraded.
System AI (autonomous actor)	A non-human, cron-driven actor that monitors thresholds, queries the LLM API, updates Product.dynamicThreshold, drafts purchase orders, and generates supplier emails — every action is logged for auditability.

2.4 General Constraints

- A modern browser supporting ES2020+ JavaScript and CSS3 is required; interactive functionality depends on client-side JavaScript.
- Authentication is fully delegated to Clerk — if Clerk is unavailable, no sign-in, registration, or authenticated action can occur.
- Tenant identity is always derived server-side via getAuthorizedOrgId(); client-supplied organization IDs are never accepted.
- FREE-tier organizations are capped at 3 products, 15 transactions/day, and 10 stock updates/day, enforced by canPerformAction() and reset nightly at 00:00 UTC.
- System availability is bounded by the provisioned PostgreSQL instance; composite unique constraints protect data integrity.
- All mutating requests are rate-limited to 100 requests per 10-second sliding window per user via Upstash Redis, failing open if Upstash is unreachable.
- File uploads are limited to image types up to 4 MB via UploadThing middleware.
- The AI subsystem depends on an external LLM API (OpenAI or Groq); on failure it logs the error and preserves the last known dynamicThreshold rather than blocking core operations.
- All LLM calls must use strict JSON-mode output (response_format: {type: "json_object"}) so responses are deterministically parseable.

2.5 Assumptions and Dependencies

Assumptions

- Monetary fields (costPrice, sellingPrice) are stored as PostgreSQL Decimal and serialized to JavaScript numbers before transmission.
- The Clerk webhook signing secret is correctly configured to validate every inbound identity event.
- The PostgreSQL instance is provisioned with adequate storage, memory, and network throughput.
- The hosting platform (Vercel or equivalent) supports scheduled cron invocation of /api/cron/* endpoints.
- Valid Razorpay keys/plan IDs and an OpenAI or Groq API key are present in the production environment.

Dependencies

The application depends on 35+ npm packages, principally: Next.js 16, React 19, Prisma 7 with @prisma/adapter-pg, @clerk/nextjs, TanStack Query v5, Zustand, Tailwind CSS v4, Zod, Razorpay, Resend, UploadThing, @upstash/ratelimit + @upstash/redis, Recharts, PapaParse, jsPDF, next-themes, the OpenAI SDK, and the Groq SDK.

Deployment requires the following secrets to be present as environment variables: DATABASE_URL, Clerk publishable/secret keys and webhook secret, Razorpay key ID/secret/plan ID and webhook secret, RESEND_API_KEY, UPLOADTHING_TOKEN, Upstash Redis REST URL/token, CRON_SECRET, and OPENAI_API_KEY or GROQ_API_KEY.

3. Specific Requirements

3.1 Functional Requirements

Requirements are grouped by module. Each entry names the responsible feature, its access rule, and its expected behaviour.

3.1.1 Authentication & Identity

ID	Feature	Behaviour
FR-1.1	User Registration	Clerk-hosted sign-up dispatches a user.created webhook; the system verifies its Svix signature and creates a local User record.
FR-1.2	User Login	Clerk manages session tokens in HttpOnly cookies; clerkMiddleware protects all /dashboard(.*) routes.
FR-1.3	Protected Routes	Unauthenticated requests to protected routes receive a 302 redirect to sign-in.
FR-1.4	User/Org/Membership Sync	Webhooks upsert or delete local User, Organization, and OrganizationMember rows as Clerk events fire; ensureUserSynced() is a lazy fallback.
FR-1.5	Org Resolution	getAuthorizedOrgId() derives the tenant from the Clerk session, auto-provisioning a Personal Workspace when no organization is active.

3.1.2 Product Catalog Management

ID	Feature	Behaviour
FR-2.1	Create Product	Admin-only; Zod-validated; atomic transaction creates the product plus an initial StockTransaction, LowStockAlert (if applicable), and AuditLog entry; blocked past the FREE-tier product cap.
FR-2.2	Dashboard Metrics	Aggregated totals (products, items in stock, low-stock count, currency), cached with tag-based invalidation and a 1-hour fallback revalidation.
FR-2.3	Inventory List	Cursor-paginated (default 25), filterable by search, stock status, and category.
FR-2.4	Update Product	Admin-only; cross-tenant check; atomic update with alert recalculation and an AuditLog entry.
FR-2.5	Delete Product	Admin-only; cascades to related transactions, alerts, adjustments, and barcodes; logs the deleted name/SKU.
FR-2.6	Bulk Delete	Admin-only; single AuditLog entry records the count and details of all deleted items.

3.1.3 Stock Transactions

ID	Feature	Behaviour
FR-3.1	Adjust Stock (single)	Members are routed to a PENDING approval; Admins/Personal users apply the change atomically with a negative-stock guard.
FR-3.2	Bulk Adjust Stock	Up to 50 line items applied in one atomic transaction with per-item negative-stock guarding.
FR-3.3	Transaction History	Cursor-paginated, filterable by product/SKU search and IN/OUT type.
FR-3.4	Recent Activity Feed	Returns the latest 5 transactions with product and user detail for the dashboard.

3.1.4 Approval Workflow

ID	Feature	Behaviour
FR-4.1	List Pending	Returns all PENDING InventoryAdjustment records with requester and product context.
FR-4.2	Approve	Admin-only; applies the stock change, logs a StockTransaction attributed to the requester, and re-evaluates low-stock alerts.
FR-4.3	Reject	Admin-only; marks the record REJECTED with no stock impact.

3.1.5 Category, Barcode & Audit

ID	Feature	Behaviour
FR-5.x	Category CRUD	Organization-scoped; deletion is blocked while a category still has associated products.
FR-6.x	Barcode CRUD	Admin-only; supports EAN/UPC/QR with a @@unique(productId, code) constraint.
FR-7.1	Audit Log Query	Paginated, searchable by user/entity, filterable by action and entity type; every CREATE/UPDATE/DELETE is captured with a JSON diff.

3.1.6 Webhooks & Subscription Billing

ID	Feature	Behaviour
FR-8.x	Developer Webhooks	Admin-managed endpoints; dispatch is fire-and-forget HTTP POST, HMAC-SHA256 signed when a secret is set.
FR-9.1	Create Subscription	Initiates a 12-month recurring Razorpay subscription.
FR-9.2	Verify Payment	HMAC-SHA256 signature check on payment_id|subscription_id before upgrading to PRO.
FR-9.3	Cancel Subscription	Cancels the Razorpay subscription and downgrades the organization to FREE.
FR-9.4	Razorpay Webhook	Handles authenticated/activated (upgrade) and cancelled/halted (downgrade) lifecycle events.

3.1.7 CSV Import/Export & Scheduled Jobs

ID	Feature	Behaviour
FR-10.1	Import Inventory CSV	PapaParse-based; auto-creates categories; SKU-based upsert; respects FREE-tier quota.
FR-10.2	Import Transaction CSV	Matches by SKU; historical only — does not mutate live quantities.
FR-11.1	Reset Daily Usage	Cron at 00:00 UTC resets dailyTxCount/dailyUpdateCount for every organization.
FR-11.2	Low-Stock Digest	Cron aggregates below-threshold products per organization and emails a consolidated HTML report.

3.1.8 AI Predictive Engine & Autonomous Procurement

ID	Feature	Behaviour
FR-15.1	Daily Demand Forecast	Cron computes 30-day sales velocity per product, sends a structured JSON prompt to the LLM in strict JSON mode, and receives suggestedThreshold, reasoning, and a confidence rating; the value is validated and written to Product.dynamicThreshold with an AIPredictionLog entry.
FR-15.2	Manual Forecast Trigger	Admin-only on-demand evaluation of a single product, returning the reasoning directly to the UI.
FR-16.1	Autonomous PO Draft	When stock ≤ dynamicThreshold and a supplier is linked, the system computes an order quantity, asks the LLM to draft a supplier email asynchronously, and creates a DRAFT PurchaseOrder.
FR-16.2	Manual PO Creation	Admin-only; if no email body is supplied, the LLM drafts one.
FR-16.3	Approve & Dispatch PO	Admin approval moves DRAFT→SENT and emails the supplier via Resend.
FR-16.4	PO Status Lifecycle	Legal transitions: DRAFT/SENT → FULFILLED or CANCELLED.
FR-17.x	Supplier Management	Admin-only CRUD for supplier contacts, linked to products via a nullable supplierId foreign key.

3.2 External Interface Requirements

3.2.1 User Interfaces

Page	Key Elements
Landing (/)	Marketing hero, feature/benefit cards, onboarding tutorial, dynamic call-to-action for authenticated vs. anonymous visitors.
Dashboard	KPI widgets, 30-day analytics chart, recent-activity feed, integration/system-status panel.
Inventory	Debounced search, status/category filters, infinite-scroll table, per-row actions, bulk selection, CSV import/export, Add-Product modal.
Transactions	Paginated ledger with search and IN/OUT filter; CSV import/export.
Approvals	Pending-request queue with Approve/Reject actions (admin-only).
Audit Trail	Searchable, filterable log with an expandable JSON diff per row.
Suppliers	Admin-only CRUD table with linked-product counts.
Procurement	PO table with status badges, AI/Manual indicator, and an expandable email-draft preview.
Settings	Categories, Organization profile, Webhooks, and Billing sub-pages.

Shared UI elements: a dark/light/system theme toggle, non-blocking toast notifications, a notification bell showing unresolved low-stock alerts, modal overlays for all CRUD flows, and skeleton/loading states on every data-heavy route.

3.2.2 Software Interfaces

Mutations are exposed as Next.js Server Actions grouped by domain: Inventory, Approvals, Settings, Webhooks, Barcodes, Import, Suppliers, Procurement, and the AI Engine. External integrations are exposed as REST API Routes:

Endpoint	Auth	Purpose
POST /api/webhook/sign-up	Svix HMAC	Clerk user/org/membership webhook receiver.
POST /api/webhook/razorpay	HMAC-SHA256	Razorpay subscription lifecycle webhook.
POST /api/subscriptions/create	Clerk	Initiates a Razorpay subscription for PRO upgrade.
POST /api/subscriptions/verify	Clerk	Verifies the Razorpay payment signature.
POST /api/subscriptions/cancel	Clerk	Cancels the active PRO subscription.
GET /api/cron/reset-usage	Bearer	Resets daily FREE-tier usage counters.
GET /api/cron/digest	Bearer	Sends the daily low-stock email digest.
GET /api/cron/ai-forecast	Bearer	Runs the daily AI demand-forecasting job.
POST /api/uploadthing	Clerk	Handles authenticated file uploads.

3.2.3 Hardware & Communication Interfaces

No specialized hardware is required; the system targets standard cloud infrastructure (Vercel for compute, a managed PostgreSQL provider for storage). All client-server traffic uses HTTPS — Server Actions ride Next.js's internal RPC protocol while API Routes follow REST conventions. Outbound developer webhooks and inbound Clerk/Razorpay webhooks are HMAC-signed HTTP POSTs. The application communicates with PostgreSQL over the standard wire protocol (port 5432) and with Resend, Upstash Redis, UploadThing, Razorpay, and the OpenAI/Groq APIs over HTTPS.

3.3 Performance Requirements

Metric	Requirement
RSC page load	< 200 ms server processing time (excluding network), via Next.js streaming.
Cached reads (95th pct.)	< 100 ms through unstable_cache with tag-based invalidation.
Uncached reads	< 300 ms.
Write operations	< 500 ms including validation, rate-limit check, $transaction, and cache invalidation.

Metric	Requirement
Rate-limit check	< 10 ms via Upstash global edge; 100 requests / 10 s per user.
Pagination	Cursor-based, 20–25 items per page, using a take+1 strategy — no total-count queries.
Cache invalidation	Tag-based (inventory-{orgId}, categories-{orgId}) plus a 1-hour revalidation fallback.
AI API latency	1–5 s acceptable inside cron jobs only; always asynchronous and non-blocking during user-facing operations.
Concurrency	Elastic serverless scaling (Vercel) with Prisma connection pooling via @prisma/adapter-pg.

3.4 Design Constraints

- Frontend must use Next.js v16 App Router with Server Components by default and Client Components marked "use client".
- Backend mutations must use Server Actions ("use server"); external integrations must use API Routes.
- The database must be PostgreSQL, accessed exclusively through Prisma v7 with the @prisma/adapter-pg driver.
- Authentication, sessions, organizations, and RBAC must be delegated entirely to Clerk — no custom auth implementation.
- Client UI state must be managed with Zustand; server-state caching must use TanStack Query.
- Every server-side input must be validated with a Zod schema before reaching the database.
- Styling must use Tailwind CSS v4 utilities with class-variance-authority for variants and a cn() helper for class merging.
- All code must be strict-mode TypeScript using ES Modules, with CUID primary keys throughout.
- All AI/LLM logic must be isolated under src/lib/ai/ and invoked in strict JSON mode; AI cron endpoints must authenticate with the same CRON_SECRET bearer-token pattern as other cron routes.

3.6 Other Requirements

Security

- No passwords are stored locally; sessions rely on Clerk's HttpOnly secure cookies.
- All webhook sources are signature-verified: Clerk via Svix HMAC, Razorpay via HMAC-SHA256, and outbound developer webhooks via a per-endpoint HMAC-SHA256 secret.
- Tenant isolation is enforced server-side; cross-tenant access attempts are explicitly rejected.
- Rate limiting fails open (rather than blocking traffic) if Upstash is unreachable, avoiding a single point of failure.
- Prisma's parameterized queries prevent SQL injection, including in the raw-SQL digest cron.
- Negative stock is prevented by an atomic $transaction rollback whenever a post-adjustment quantity would fall below zero.
- AI outputs are validated (positive-integer threshold) before being written to the database; malformed responses are discarded, not applied.

Reliability, Availability & Scalability

The system targets 99.5% uptime on Vercel's serverless infrastructure. Prisma connection pooling and a cached global client prevent connection exhaustion. Webhook dispatch is fire-and-forget so external failures cannot crash the application. All multi-step mutations are wrapped in Prisma $transaction blocks for atomicity, and AI forecasting failures are isolated per product so one failure never blocks forecasting for the rest of the catalog. Cursor-based pagination and tag-invalidated caching keep read performance consistent as tenant and dataset counts grow, and the multi-tenant schema supports unlimited organizations without structural change.

Localization & Currency

All timestamps are stored in UTC and converted to the viewer's local time client-side via date-fns. Organization currency is configurable (default INR, ISO-4217, 3 characters); monetary fields are stored as PostgreSQL Decimal for precision.

4. Appendices

Appendix A — Database Entity Schema (Prisma)

The system persists state across 15 Prisma models in PostgreSQL, all scoped by organizationId for tenant isolation.

Entity	Key Fields	Relations
User	id (CUID, from Clerk), email (unique), name, avatarUrl	OrganizationMember, StockTransaction, AuditLog, InventoryAdjustment (x2)
Organization	id, name, slug (unique), currency, subscriptionTier, dailyTxCount, dailyUpdateCount, razorpaySubId (unique)	OrganizationMember, Category, Product, AuditLog, WebhookConfig, Supplier, PurchaseOrder
Product	id, organizationId, categoryId, sku (unique/org), quantity, costPrice, sellingPrice, lowStockThreshold, dynamicThreshold, supplierId?	Organization, Category, StockTransaction[], LowStockAlert[], InventoryAdjustment[], Barcode[], Supplier?, PurchaseOrder[], AIPredictionLog[]
StockTransaction	id, productId, userId, type (IN/OUT), quantity, notes	Product (Cascade), User
InventoryAdjustment	id, productId, requestedById, approvedById, quantityChange, status	Product (Cascade), User (x2)
Category	id, organizationId, name, description	Organization, Product[]
LowStockAlert	id, productId, triggeredAt, resolvedAt, notified	Product (Cascade)
Barcode	id, productId, type (EAN/UPC/QR), code (@@unique with productId)	Product (Cascade)
WebhookConfig	id, organizationId, url, secret, events[], isActive	Organization (Cascade)
Supplier	id, organizationId, name, email, phone?	Organization (Cascade), Product[], PurchaseOrder[]
PurchaseOrder	id, organizationId, supplierId, productId, quantity, status, generatedBy, emailBodyDraft	Organization, Supplier, Product (all Cascade)
AIPredictionLog	id, productId, previousLimit, newLimit, reasoning	Product (Cascade)
AuditLog / EmailNotification	organizationId, userId, action/eventType, JSON payload	Organization, User

Key constraints: Product @@unique([organizationId, sku]); Barcode @@unique([productId, code]); Organization.slug and Organization.razorpaySubId unique; User.email unique. All foreign-key columns are indexed for query performance.

Appendix B — Project Directory Structure

```
swiftstock/
├─ prisma/
│  └─ schema.prisma
├─ src/
│  ├─ middleware.ts
│  ├─ app/                          # Strictly routing & pages
│  │  ├─ (auth)/
│  │  ├─ (dashboard)/
│  │  ├─ api/
│  │  ├─ layout.tsx
│  │  └─ page.tsx
│  ├─ features/                     # ⬅ The game changer for scalability
│  │  ├─ inventory/
│  │  │  ├─ actions.ts              # All inventory-related server actions
│  │  │  ├─ components/             # Inventory-specific tables, forms, charts
│  │  │  └─ validations.ts          # Zod schemas for inventory
│  │  ├─ procurement/
│  │  │  ├─ actions.ts
│  │  │  └─ components/
│  │  ├─ audit/
│  │  │  ├─ actions.ts
│  │  │  └─ components/
```

```
│   │   └── barcodes/
│   │       ├── actions.ts
│   │       └── utils.ts
│   ├── components/                # Global UI only
│   │   ├── layout/                # Sidebar, Header
│   │   └── ui/                    # Buttons, Inputs, Dialogs
│   └── lib/                       # Global configs & clients
│       ├── ai/
│       ├── auth.ts                # Renamed from auth-helpers.ts
│       ├── db.ts                  # Renamed from prisma.ts
│       ├── ratelimit.ts
│       ├── uploadthing.ts
│       └── utils.ts
```

Appendix C — System Use Case Workflows

ID	Use Case	Flow Summary
UC-1	Admin Adds Product & Adjusts Stock	Admin submits the product form → Zod validation → atomic create + audit log → admin adjusts stock → transaction logged, alerts evaluated.
UC-2	Member Requests Stock Adjustment	Member submits a change → PENDING InventoryAdjustment created → admin reviews → approve (stock updated, transaction logged) or reject.
UC-3	PRO Subscription Upgrade	Admin initiates checkout → Razorpay subscription created → payment completed → HMAC signature verified server-side → organization upgraded to PRO.
UC-4	Low-Stock Alert & Webhook Dispatch	Adjustment drops quantity ≤ threshold → LowStockAlert + EmailNotification created → Resend email sent (PRO) → active webhooks receive a signed HTTP POST.
UC-5	Autonomous Procurement & AI Forecasting	Daily cron → LLM analyzes 30-day velocity → dynamicThreshold updated → if stock is below threshold and a supplier is linked, a draft PO is created → admin approves → Resend emails the supplier.

5. Index

Term	Section(s)
AI Prediction Log / Dynamic Threshold	1.3, 2.2, 3.1.8, Appendix A
Approval Workflow	2.2, 2.3, 3.1.4, Appendix C (UC-2)
Audit Log	2.2, 3.1.5
Authentication (Clerk)	2.1, 2.4, 3.1.1, 3.4
Autonomous Procurement	2.2, 3.1.8, Appendix C (UC-5)
Barcode Management	2.2, 3.1.5
CSV Import/Export	2.2, 3.1.7
Dashboard / Analytics	2.2, 3.2.1
Demand Forecasting (AI)	1.2, 2.2, 3.1.8
Free Tier Limits	2.2, 2.4, 3.1.7
Low-Stock Alerts	2.2, 3.1.3, Appendix C (UC-4)
Multi-Tenancy	2.1, 2.4, 3.6 (Security)
Product Management	2.2, 3.1.2
Purchase Order (PO)	1.3, 2.2, 3.1.8, Appendix A, C (UC-5)
Rate Limiting	2.4, 3.3, 3.6 (Security)
Razorpay / Billing	2.1, 3.1.6, Appendix C (UC-3)
Stock Transaction	2.2, 3.1.3
Supplier Management	2.2, 3.1.8